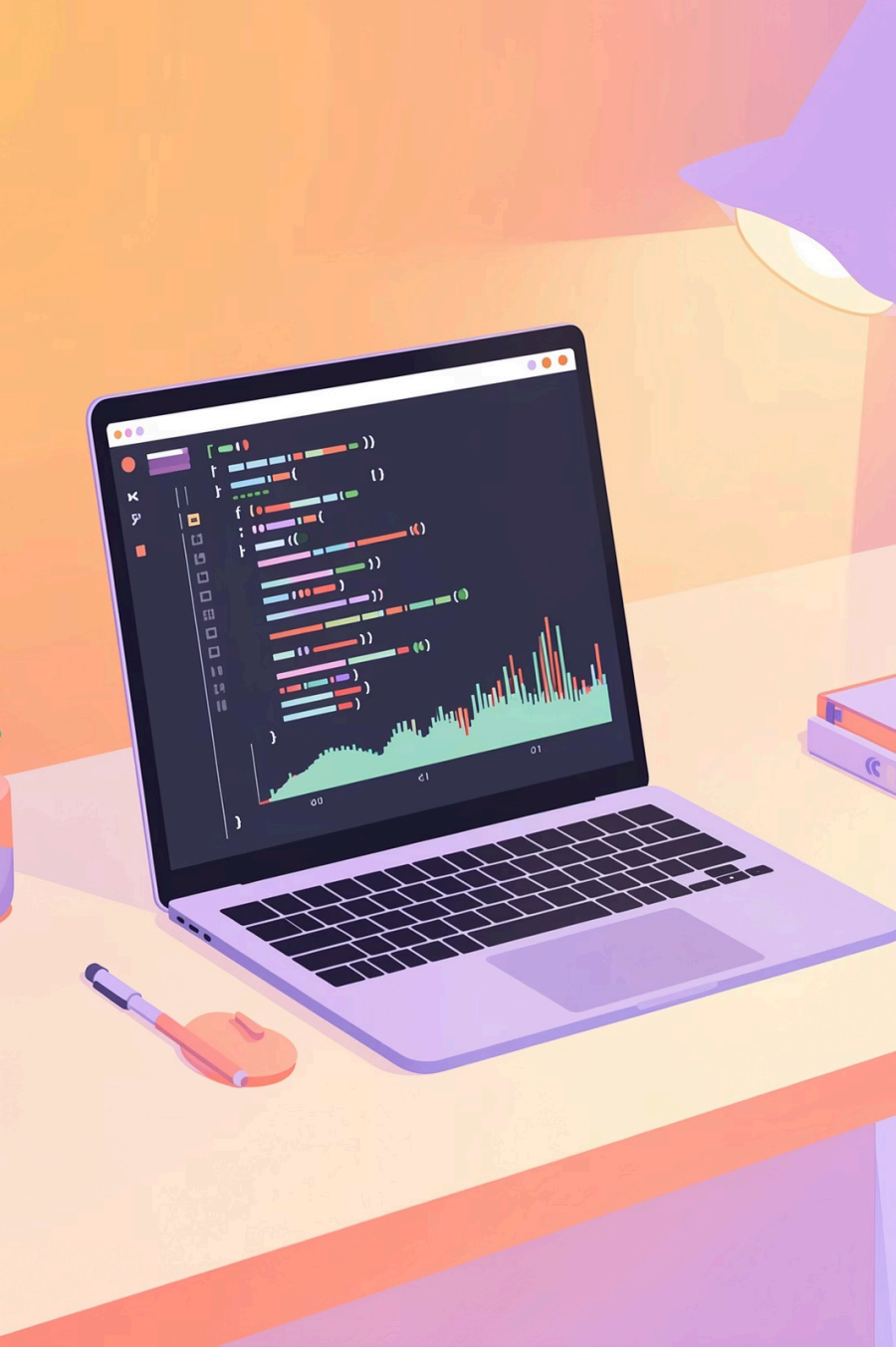




Aspect-Based Sentiment Analysis using TF-IDF and Logistic Regression

Multi-Label NLP Classification Pipeline — concise technical walkthrough for data science students

 ABSA

 TF-IDF → LR

 MULTI-LABEL

Problem Statement

We must extract aspect→sentiment pairs from noisy, mixed-language reviews where a single text can carry multiple sentiments. The target is multi-label prediction: identify which aspects are present and the sentiment for each (e.g., service: positive).

Core challenges

- Multi-label overlap within a sentence
- Code-switching (Arabic + English)
- Imbalanced labels and sparse signals

Goal

Design a reproducible pipeline that maps text → TF-IDF → probabilistic multi-label predictions with calibrated thresholds.



Dataset Overview

Split and schema used for experiments:

Sizes

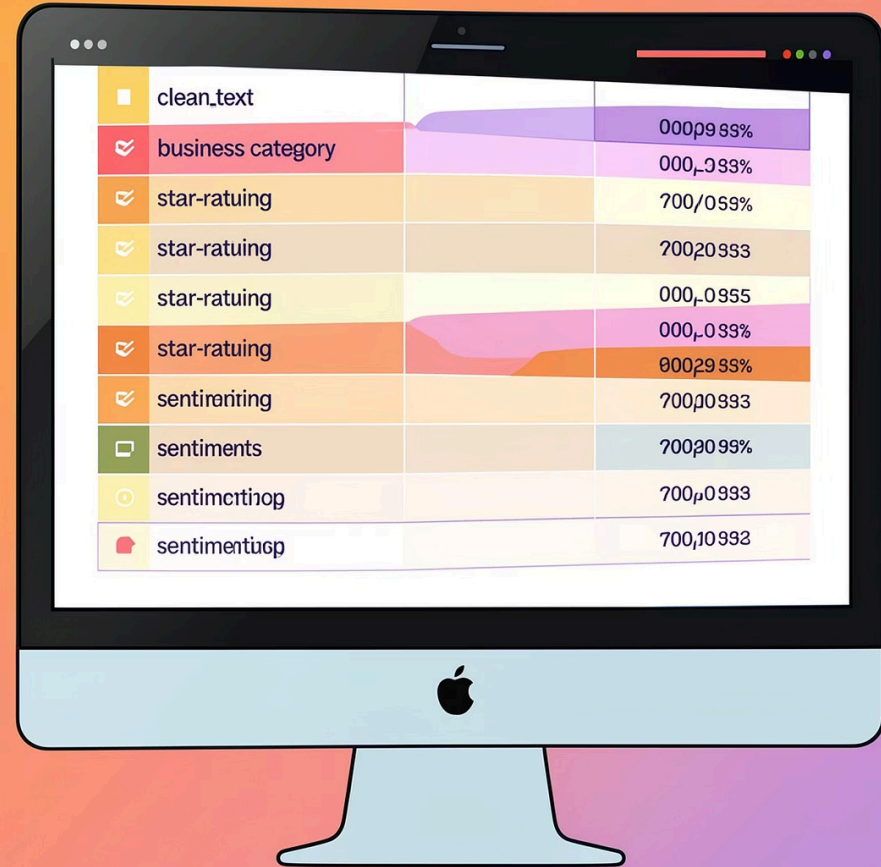
Train: 1,971 samples · Validation:
500 samples

Key columns

- clean_text
- business_category
- star_rating
- aspects + sentiments

Text characteristics

Mixed Arabic & English, informal, abbreviations, emojis and noisy punctuation.



Label Engineering

Transform aspect + sentiment pairs into unified categorical labels and encode as multi-hot vectors using MultiLabelBinarizer.

Example transformation

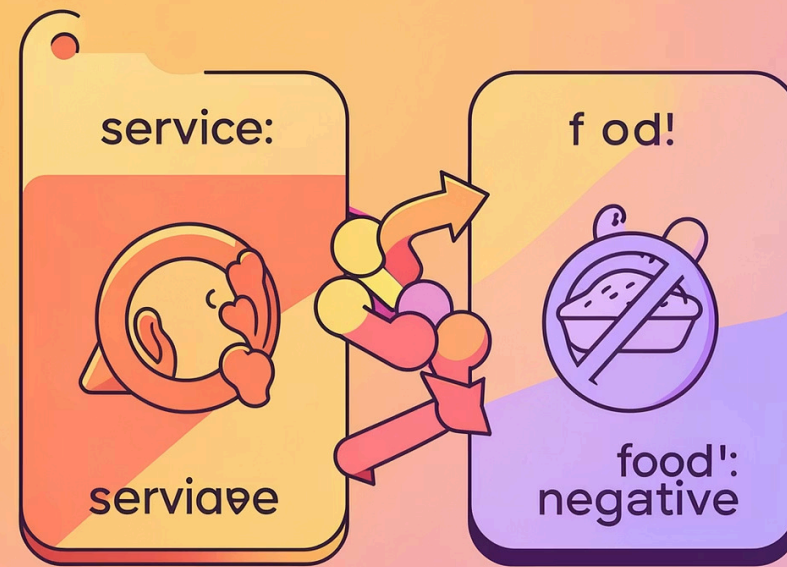
Input: "Great service but the food was cold" → Labels:

[service:positive, food:negative]

- Concatenate aspect and sentiment as label tokens
- Handle missing aspects via empty label set
- Total distinct labels: 24

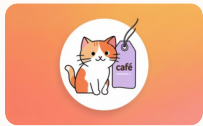
Implementation notes

Use `sklearn.preprocessing.MultiLabelBinarizer` for consistent ordering and inverse transforms for interpretability.



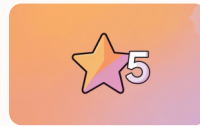
Feature Engineering

Construct a single text input combining review text with categorical tags to provide contextual signals.



Business category

Append as token: [CAT: café] — helps capture category-specific vocabulary



Star rating

Append as token: [RATING: 5] — provides coarse sentiment prior



Final input

Example: "Great service
[CAT: café] [RATING: 5]"
→ fed to TF-IDF

TF-IDF



Text Representation — TF-IDF

We use TF-IDF to convert enriched text to high-dimensional sparse features.

Configuration

- `max_features = 20,000`
- `gram_range = (1, 3)` to capture phrases
- `sublinear_tf` and `stop-word` handling recommended

Outcome

Produces a sparse feature matrix suitable for linear classifiers and efficient one-vs-rest training.



Input

TF-IDF

One-vs-Rest LR

Thresholding

Architecture: a pipeline that outputs per-label probabilities, which are thresholded into a final multi-label set.



Classifier details

Logistic Regression (One-vs-Rest): max_iter=2000 · C=3.0
· class_weight='balanced'



Probabilities

Model produces sigmoid probabilities per label for independent decisions.



Training & Prediction

Training

- Fit TF-IDF on training text
- Train One-vs-Rest Logistic Regression per label
- Use `class_weight='balanced'` to partially mitigate imbalance

Prediction pipeline

Predict probabilities on validation set → apply threshold = 0.3 → convert to multi-label outputs. Threshold chosen to favour recall in lower-frequency labels.

Evaluation Results

Quantitative performance on the validation set:

0.076

Subset Accuracy

Exact-match proportion of samples
with identical label sets

0.535

Micro F1

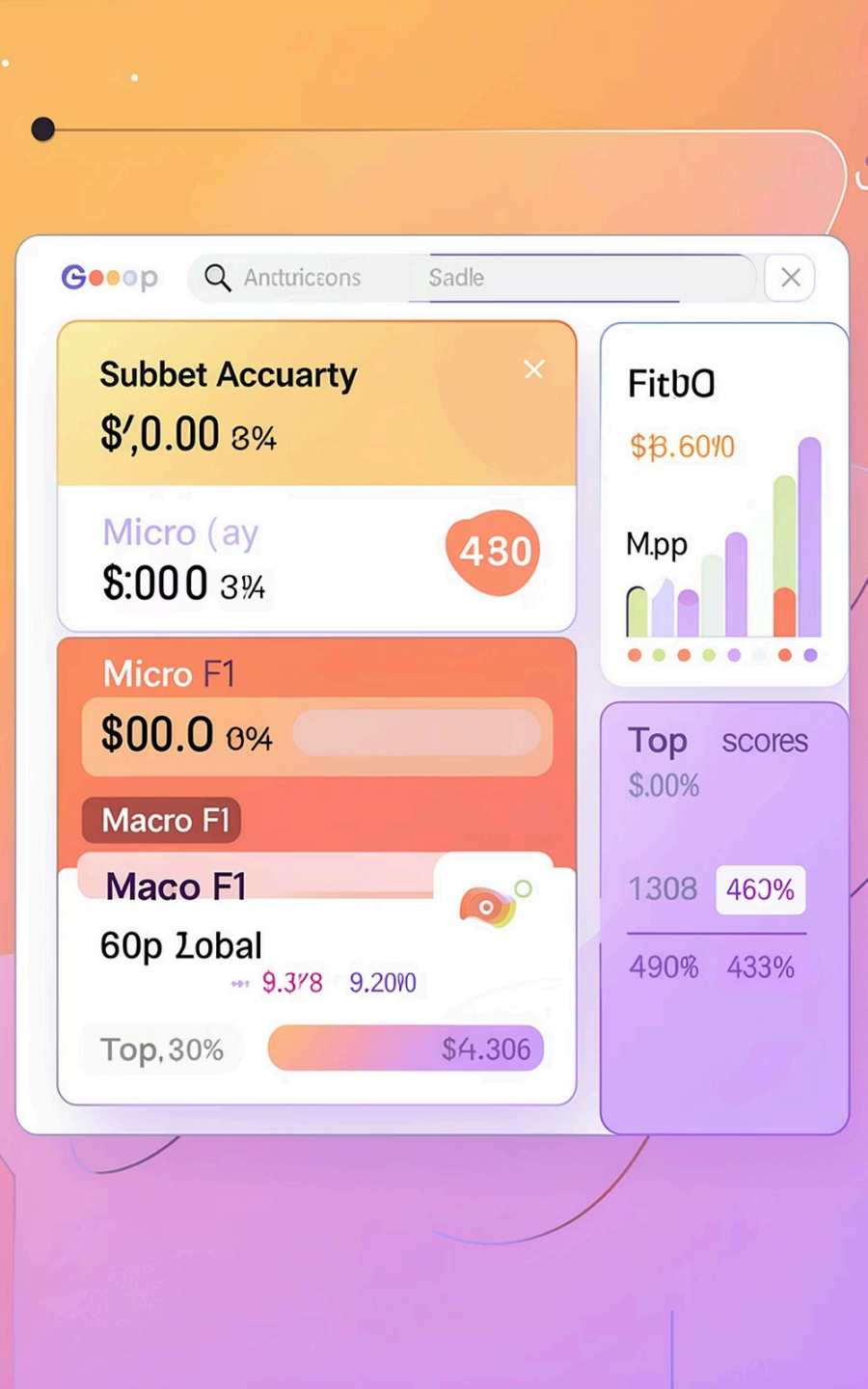
Aggregated F1 across all label
decisions (favouring frequent labels)

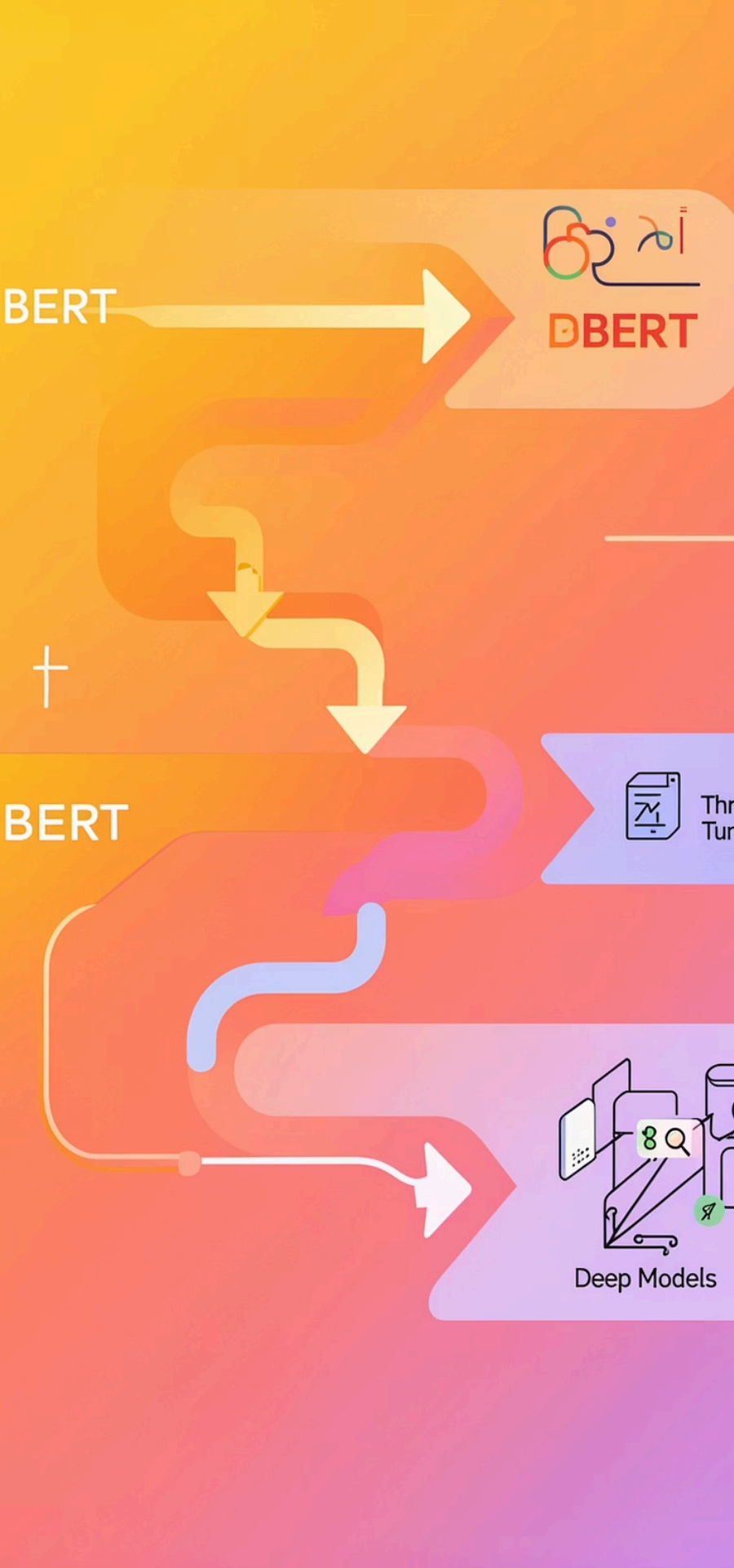
0.315

Macro F1

Average F1 across labels (treats all
labels equally)

Top per-label F1 scores: service:positive 0.739 · service:negative 0.642 ·
app_experience:negative 0.593 · ambiance:positive 0.570





Conclusions & Next Steps

TF-IDF + One-vs-Rest Logistic Regression offers a strong, interpretable baseline for ABSA. It performs well on frequent labels but struggles with imbalance and overlapping sentiments.



Key insights

Good micro-level performance; low subset accuracy due to multi-label complexity.



Immediate improvements

- Per-label threshold tuning or calibration
- Stratified sampling and stronger class-balancing



Longer-term upgrades

- Transformer embeddings (e.g., multilingual BERT) to handle code-switching
- Neural multi-label architectures and sequence tagging variants

- ❏ For reproducibility: log preprocessing steps, TF-IDF vocabulary, label order from MultiLabelBinarizer and the per-label thresholds used at inference.